ReActive is All PA Need

个人助理需要的，只是一个会“听”的 ReAct

Xuanhao Hu

Tsinghua University　2026 年 10 月

本文为作者独立撰写的研究性文章，与 DeepSeek 及其产品没有直接关系，也不代表 DeepSeek 的立场。

个人助理（Personal Assistant，PA）的瓶颈早已不在单步推理，而在“反应”：能否被环境唤醒、能否在被打断后仍然连贯、能否在同一个情境里不必把同一件事想第二遍。一篇公开文章提出的 Super-agent 架构已经把“如何组织一个持续在线的 Agent”回答得相当完整，本文保留这套架构，只打两个补丁。

补丁一是 **MSP**（Model Sensing Protocol），解决通用性：一切皆节点、一切皆信号，任何能力与任何事件都以同一形态接入，于是 ReAct 的观察可以从“伸手调用工具”变为“订阅信号推送”。补丁二是 **Mem + Exp + SKILL**，解决习惯：经验 Exp ∈ Mem + SKILL 是记忆与技能经后台整理后的精华，在触发时注入提示词，并且明确地未验证——它是待检验的假设，不是事实。前者让反应通用，后者让反应有习惯。

本文仍是一篇面向工程实践的架构思路探讨，并非对 DeepSeek Harness 现有产品功能的介绍，**与 DeepSeek 及其产品没有直接关系**：文中的基线架构叙述基于公开资料，两个补丁与全文写作由作者独立完成。MSP 已完整实现；Mem/Exp/SKILL 是实验性原型，7.3 节把“已完成”与“实验性”逐项分开。

---

1. Super-agent：一个已经很好的起点

过去一年里，以 Coding Agent 为代表的单任务 Agent 已经证明，模型能够使用工具解决复杂问题，且越来越少需要人类持续监控和逐步干预。与此同时，模型的多 Agent 协同能力也在提升，使跨会话的组织与调度变得可行。随着人类在底层执行中的参与减少，人的角色正逐步上移到更高层的决策与验收。

模型能力的这一提升，也催生了新的应用形态。近期个人助手（Personal Agent）开始集中涌现，它们直接接入日常的消息、日程、邮件与工作平台，持续观察信息流并协助处理事务。这种持续在线、事件驱动的工作模式并不局限于个人场景，个人助手只是其中一类应用；在软件开发等商业场景中，它同样很有价值。

然而，现有的主流 Agent 设计还难以在多任务和团队协作的规模上支撑这一模式。Agent 的工作模式仍然以单一 Session 为基本单位，不同 Session 之间彼此独立。当用户并行推进多项工作时，必须手动切换并记住各个 Session 的上下文，而据我们观察，一个人通常只能同时跟进 3

到 5 个活跃 Session。在这种设计下，单任务的上限取决于模型能力，多任务的上限取决于人的注意力。

如果只在现有的本地单会话 Agent 上借助 CLI、MCP 或外部连接器做简单扩展，Agent 虽然也能接入外部消息，但仍受制于单会话同步阻塞、多任务上下文混乱、本地设备离线即停止运行等问题，需要大量人工维护。

要突破这些限制，需要打破 Agent 与单次手动会话的绑定，使其成为一个能跨会话保持上下文、同时推进多项复杂任务且互不干扰的连续实体。我们把这种 Agent 形态及其系统架构称为 **Super-agent**。

例如，在一个基于 DeepSeek Harness 运行的 Super-agent 实例中，同一个开发任务可以在 IM 和 GitHub 之间连续推进；这个实例还会持续观察公共群聊中的反馈，在无人 @ 的情况下主动定位根因并提交修复 Pull Request：



同一个任务在 IM 群聊与 GitHub Pull Request 评审之间连续推进



同一张图“个人版”：一个任务跨 IM、日历、邮件三个信道连续推进，用户只在 IM 里说了一句话。差别不在流程，而在能力从哪来——天气是一条被订阅的信号，日历与邮件各是一个节点，助理没有为它们写任何专用连接器（3.5 节）。



持续关注公共群聊，在无人 @ 的情况下主动识别缺陷并提交修复 Pull Request

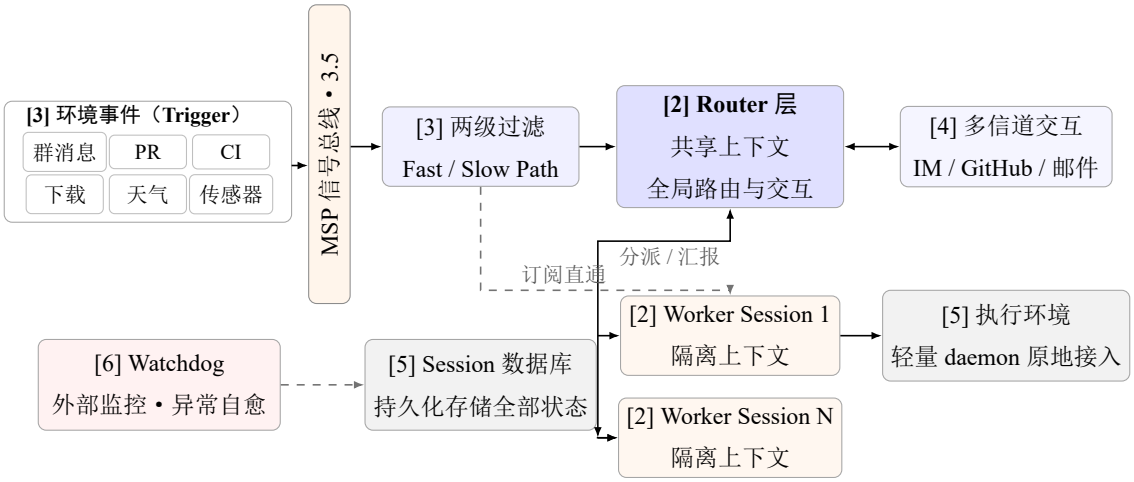
具体来说，我们认为 Super-agent 具备以下四个基础特征：

- **连续**：用户可以直接在 IM 聊天、GitHub 评论等熟悉的工作界面与它交互，感觉始终只存在一个 Agent，不需要去想“Session X 的上下文中是否有 Y 信息”。例如，用户先在 IM 上提出采用方案 A、随后在 GitHub Pull Request 评论里改为方案 B，它能理解并调整执行方向。
- **高效**：交互响应迅速，同时能并行、高质量地完成多个复杂任务。无需等上一个命令或任务完成，就能接受下一条指令。
- **主动**：无需等待用户逐条下达指令，能持续观察环境事件并主动推进值得处理的事项，不需要介入时则保持安静。
- **持久**：始终在线、直接接入用户真实的工作环境，并在长期运行中遇到异常时自动恢复。

单个 ReAct Loop 无法同时支持上述四个特征，底层需要一套相应的架构。相似的设计约束会导出相似的架构选择。我们认为，无论是面向个人的助手，还是面向软件开发等商业场景的 Agent，如果目标是构建一个持续在线、并发响应环境事件的系统，那么在底层就可能会采用相近的架构。同时，这一过程中面临的许多问题与传统计算机系统长期探讨的问题具有很高的相似性，因此计算机系统中的许多经典设计思想在这里同样适用。

在具体落地中，这套架构会有不同的变体，但我们认为以下五个模块构成了支撑上述四个基础特征的基本框架：通过双层架构（第 2 节）兼顾**连续**交互与**高效**并发，通过事件驱动（第 3 节）实现**主动**响应，通过面向人的交互（第 4 节）在交互过程中同时做到**连续**与**高效**，并通过计算、状态与环境的分离（第 5 节）及 Watchdog（第 6 节）实现跨设备的**持久**在线与异常自愈。

下图展示了 Super-agent 的整体系统架构，图中各模块的标号 [2] 至 [6] 分别对应下文第 2 至第 6 节：



Super-agent 整体系统架构：标号 [2] 至 [6] 对应第 2 至第 6 节；[3] 环境事件里的六类来源只是节点，经 MSP 信号总线（3.5 节）以同一形态进入系统；Worker 可越过 Router 直接订阅 Thread。

前面五节回答的是“怎么把系统组织起来”。在往下讲的过程中，我们在这套架构上打两个补丁，补上它没有回答的两个问题：**2.3 节**补上习惯——用 Mem + Exp + SKILL 把过去的判断整理成经验，在触发时注入，使助理不必把同一件事想第二遍；**3.5 节**补上通用性——用 MSP 把“一切能力”与“一切事件”统一成节点与信号，使接入从工程适配变成协议约定。7.3 节交代两者的实现程度：MSP 已完整落地，经验这一层是实验性的。

两个补丁共享同一个前提：系统的感知必须能“推”，而不只是被“拉”。这也是本文把这种形态命名为 **ReActive** 的原因——把词拆开看，是 *Re*asoning、*Act* 与 *ively*：推理、行动，并且主动地。ReAct 给了前两半，第三半是本文要补的：行动必须由环境来发起，而不是由指令来排程。合起来，才是个人助理所需要的全部。

顺便说清标题里的 **PA**。它通常读作 *Personal Assistant*（个人助理），但首字母的那个 P 也可以读作 *Proactive*——*Proactive Assistant*，主动式助理。本文并不打算在两个 P 之间选一个，因为这两个 P 恰好就是本文的两半：**Personal** 说的是为谁服务、在哪里服务——跨信道的连续交互，会话对用户透明（第 2、4 节）；**Proactive** 说的是怎么服务——不等指令，被事件唤醒，该介入时介入（第 3 节，以及 3.5 节把它落到协议上）。A 则是同一个词：Assistant，或者 Agent，两种写法在本文里指的是同一样东西。

换句话说，一个好的个人助理本来就是主动的；把它写成“个人助理”还是“主动式助理”，只是从哪个侧面描述同一件事。标题里的那个双关，指的就是这里：PA 的 P 有两个，而它们本不该被分开。

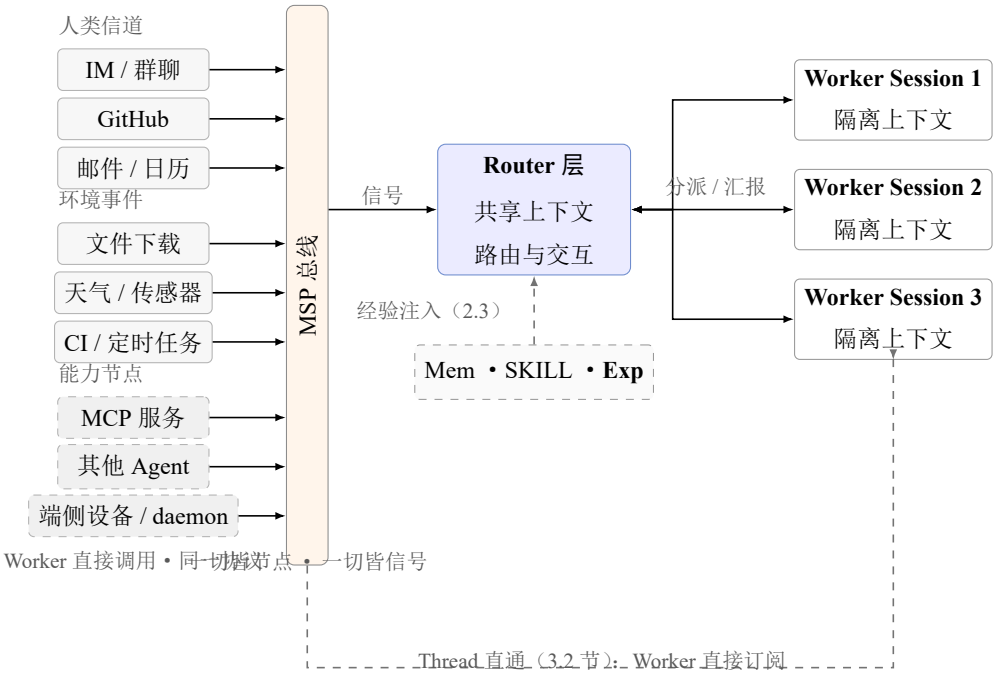
2. 双层架构

要同时实现跨信道的**连续**交互与多任务的**高效**并发，系统在底层会话组织上面临连续性与吞吐量之间的根本权衡（Trade-off）。

如果每条事件都交给同一个 Session 处理，虽然上下文最连续，但受限于单个 ReAct Loop 串行执行的速度上限，任务完成耗时长、吞吐低，且多任务的中间细节混杂在一起也会影响完成效果。

反过来，如果每条事件都直接新开独立的 Session 去处理，虽然并发吞吐最高，但一旦任务之间并不完全独立，缺少全局上下文会让各个 Session 的结果彼此割裂，同时多个 Session 各自向用户响应也会造成交互混乱。

为了兼顾连续性与吞吐量，我们可以将“维护全局连续性”与“高并发执行”拆分到两个不同的层级中：由常驻的 **Router 层**（2.1 节）统一承接人类交互与全局协调，由上下文隔离的 **Worker 层**（2.2 节）并发执行具体任务，如下图所示：



双层架构：Router 层统一负责路由与对外交互，多个 Worker Session 隔离并发执行；左侧三类节点——人、环境、被调用的能力——语义不同，接法相同，都挂在同一条 MSP 总线上；Worker 可越过 Router 直接订阅 Thread，Router 的判别由 Mem/Exp 提供先验。

通过这一双层架构，系统在底层接管了上下文管理与并发执行，从而在用户侧淡化了 Session 的存在，使用户面对的始终是同一个 Super-agent。从这个角度看，让用户手动管理 Session，在很大程度上是从底层 LLM API 延续下来的设计惯性；Session 应当是系统的实现细节，而不应是用户的认知负担。

在现有设计中，Session 同时承载了技术上的上下文管理与面向用户的项目管理。将产品形态与底层 Session 解耦后，面向用户的项目管理便不再受 Session 的限制，向用户暴露怎样的交互与任务管理方式成为了一个独立的产品问题。我们将在后文讨论这个问题。

这里的“双层”是逻辑上的职责划分，每一层在实现上都不必局限于单个 Session，而是可以有各自的内部结构。下文将分别介绍 Router 层与 Worker 层的设计。

2.1 Router 层

Router 层是常驻的全局路由层，负责承接所有入口的人类交互。所有跨任务的信息与决策都在 Router 层中汇总，由它充当全局的序点（Sequence Point）、协调点和记忆。为了避免这一全局节点成为系统的性能瓶颈，Router 层在设计上不直接执行具体任务，只负责人类交互与任务分派，甚至可以不配备 Bash 工具。

这种由一个中心节点分派多个执行节点的结构，表面上看与常见的 Agent Team 相似，但两者的侧重点不同。常见的 Agent Team 主要用于单个复杂任务内部的步骤拆解与角色协作，由于子步骤通常耦合较紧，Team Lead 往往需要深入理解任务细节、规划子任务并高频协调各个成员，自身承担了很重的推理负载。而在 Super-agent 中，Router 层只处理任务之间的高层关联与分派，无需介入单任务内部的拆解；相反，如果通过提示词约束一个 Agent Team 的主会话不亲自执行具体任务，仅负责对外交互并将不同任务分派给各成员独立推进，那么它也可以视为这种双层架构。从我们的实践来看，Router 层的任务常常没有想象中复杂，使用较轻量的模型往往就能兼顾速度与效果。

在内部实现上，Router 层同样不必局限于单个 Session，甚至部分逻辑未必需要调用 LLM。我们选择了最直接的单个常驻 LLM Session 作为 Router 层的核心实现，并发现它并没有表面看起来那么容易成为吞吐瓶颈，因为上一轮推理期间到达的环境事件（即第 3 节的 Trigger）会积压下来、在下一轮一并处理，形成**批处理（Batching）**，而合并处理的额外开销很小：

- 当外部事件流平稳时，Trigger 逐条进入 Router Session，此时单次响应延迟较低；
- 当事件密集涌入、Router Session 开始出现排队拥堵时，上一轮推理期间积压的多条 Trigger 会自动合并，在下一次 LLM 调用中一并注入。Router Session 只做轻量的交互与分派，每步通常是长输入、短输出。把积压的多条 Trigger 合并进一次 LLM API 调用，会在 Prefill 阶段并行计算，单次请求耗时的增长远慢于批处理大小。因此在一定范围内，积压越多，单个决定分摊的推理时间越低；上限取决于上下文长度，以及模型在一轮中能准确处理多少条 Trigger。

我们将在 7.2 节通过实验定量展示这一批处理机制的效果。加上后文将介绍的订阅与直通机制以及链式召回（见 3.2、3.3 节），单个常驻 Router Session 在保持上下文连续的同时，在我们目前的团队规模下已经够用。

当规模进一步扩大、单个会话确实不足以承载时，Router 层内部也可以设计为多层过滤结构，或**水平切分（Sharding）**为多个会话。此时，Router 层的内部设计本质上是连续性与吞吐量的权衡。切分粒度越细，Router 层的并发吞吐量就越高，但分片之间的记忆与上下文连续性就会越低。在全局记忆问题被解决之前，这一权衡始终存在。

2.2 Worker 层

Worker 层由负责具体执行的会话组成，各个 Worker 彼此上下文隔离，挂载完整的执行工具，在独立上下文中专注完成单一任务。由于 Worker Session 直接决定任务结果与代码质量，通常需要配置能力最强的模型。

Worker 不会直接与人类交互，其推理过程和工具调用不会直接输出到外部信道，而是向 Router 层汇报状态、由 Router 层统一向用户回复，从而在保障并发度与交付质量的同时避免多个会话各自回复带来的混乱。

在多任务并发执行时，各个 Worker 独立推进任务，同层 Worker 之间通常也不需要直接通信。多个 Worker 在具体执行中的冲突往往出现在同一个 Pull Request 等共享工作环境中；只要它们都订阅了对应的 Thread，就会收到相同的事件通知（见 3.2 节），从而有机会自行发现和解决冲突，少数跨任务协同则由 Router 层中转。

与 Router 层类似，每个 Worker Session 在内部也不必局限于单个会话：在执行复杂单任务时，Worker 内部可以再创建 Subagent，也可以本身就是一个 Agent Team，这些对上层的 Router 层保持透明。这一结构既适用于代码协作场景（同时推进多个独立的问题复现、Pull Request 评审与特性开发而互不阻塞），也适用于个人助手场景：虽然个人助手在界面上通常只提供一个单会话的聊天页面，但当用户在同一对话框中同时讨论多个不同话题时，前端的单一聊天页负责维持交互的连续性，而底层可以为每个话题分别分派 Worker，并行推进任务、起草回复。

2.3 判断如何沉淀：记忆、技能与经验

2.1 节说 Router 层每轮都在做判断，但没有说这些判断凭什么不必重新算一遍。本节补上这一层，它是本文两个补丁中的第二个：让“反应”有习惯。

判断的成本

在双层架构里，Router 层每一轮都在做同一类判断：这条消息要不要管、它属于哪个任务、要不要新建一个 Worker、要不要打断正在执行的工作。这些判断的质量，取决于当轮的上下文与模型的临场发挥。

问题不在单次判断的对错，而在重复。同一个人的同一类请求、同一个仓库的同一类反馈，在几周内会以几乎相同的形状出现几十次；而每一次，系统都从头推理一遍。这带来三种代价：慢（每轮都要一次完整的判别调用）、贵（同样的推理反复付费）、不稳（温度、上下文微小差异会让同类情形得到不同处理）。

在个人场景里，这类重复尤其明显：每逢降温，这个用户都会问“要不要带伞”；每次下载失败，他都会先怀疑网络代理；每个跨时区的会议，他都要确认一遍对方时区。这些判断都值得做对，却不值得每次都从头推一遍——它们既不该被写死成规则（天气一变、代理换掉，规则就错），也不该被反复推理。

人不是这样工作的。人对熟悉情境的处理不是“重新推理”，而是“认出来”——看到形状就调用一个已经形成的倾向，只有在预期被打破时才回到推理。这类倾向在认知科学里被称为

习惯化或自动化加工，其代价低、速度快，代价是可能错：习惯会固化错误的因果，也会在环境漂移后继续沿用旧判断。

我们想要的正是这种“便宜但可能错”的东西——只要系统知道它可能错。

三个对象

我们把这一层拆成三个对象，它们的分工如下表。

Mem、SKILL、Exp 的分工。三者的关键差别不在“存什么”，而在“是否可执行”与“是否已验证”。

对象	是什么	形态	失效方式
Mem	发生过什么、世界是什么	情景与语义的原始记录，可按需召回	记不住、取不到、检索到无关内容
SKILL	怎么做成一件事	声明式的过程与工具编排，可执行、可检查	过程过时、前置条件不成立
Exp	这种情境下通常该怎么做	情境、动作与预期结果的倾向，不可执行	虚假相关、分布漂移、错误固化

三者都是“过去”，但都不是“现在”：Mem 是原料，SKILL 是已经沉淀好的能力，Exp 是介于两者之间的倾向——它比 Mem 更可复用，比 SKILL 更不确定。

Exp 是 Mem 与 SKILL 的精华

我们把经验写成

$$\text{Exp} \in \text{Mem} + \text{SKILL}$$

这里用“$\in$”而不是“$\subset$”，是刻意的：Exp 既不是 Mem 的子集，也不是 SKILL 的子集，而是两者经过整理之后的像。整理不是一个筛子，而是三个算子：

- **折叠**：把多条同构的情景折成一条。二十次“用户贴一段报错、随后补一句‘刚才还能用’”可以折成一条“这类补充通常意味着最近有过变更，先看最近提交”。
- **去噪**：丢弃没有信息量的情景。失败但没有归因的记录、被人类明确否定的处理，不进入经验，只留在 Mem 里。
- **归因**：把结果归到可复用的条件上，而不是归到具体对象上。“这次因为改了 `foo.py` 所以失败”不可复用；“跨文件的接口改动后，先跑一遍契约测试”可复用。

折叠与归因都要求把具体抽象成条件，这必然丢信息——所以 Exp 天然比 Mem 更模糊。这是精华的含义：用确定性换可复用性。

已有的能力（SKILL）同样参与：一个被反复调用、且调用后结果稳定的 SKILL，会和触发它的情境一起被折叠成 Exp——“这种情况用那个技能，通常就够了”。这就是为什么 Exp 的

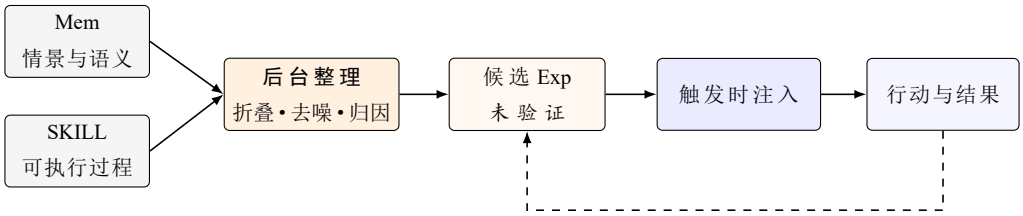
输入里必须有 SKILL：否则经验会退化成“对历史片段的相似度匹配”，而不是“在已知能力空间里的选择倾向”。

后台整理，而不是在线推理

Exp 不是实时算出来的，而是在后台整理的：不在推理路径上，不占用户等待的时间，在低峰期按任务批量进行（如下图）。

这样做有两个理由。其一是成本：折叠与归因本身需要模型参与，若放在每次交互里，等于把“省下来的推理”又花了回去。其二是质量：整理需要看到足够多的样本才能判断什么是“同构”；在线逐条整理的粒度太细，只能得到“刚才发生了什么”，得不到“这类情况通常怎样”。

整理产出的不是结论，而是候选：一条带来源、带支持计数、带置信度的假设。候选进入经验池，等待被触发、被使用、被检验。



经验的生命周期：交互留下的记录进入 Mem，被调用过的 SKILL 一同参与整理；后台整理产出候选 Exp；触发时按情境匹配注入；结果沿虚线回流，更新置信度，反复被验证者升级为 SKILL。

触发时注入：经验也应当是“推”的

经验怎么进入上下文？有两种做法。一种是拉：给模型一个“回忆经验”的工具，让它需要时自己去查。另一种是推：由触发本次任务的事件携带情境，系统在构造提示词时自动检索并注入相关经验。

我们选后者，理由与 3.5 节相同：拉的代价被转移给了模型——它得先意识到“自己可能需要经验”，才会去查；而多数时候，恰恰是不知道自己不知道的情境最需要经验。推把这件事交给系统：既然触发事件已经给出了情境（谁、在哪个信道、什么类型的事件、涉及哪个对象），检索就不需要模型参与。

注入的形式是“一小把候选”，而不是“一大段历史”：按情境相似度、置信度与新鲜度排序，取前若干条，每条压缩成一两句话，附上来源与置信度。上下文窗口是稀缺资源，经验的注入必须遵守与 Inline & Spill（第 3.3 节）相同的纪律：进窗口的是判断的形状，不是判断的原始材料。

这也让经验层与本文的主题对齐：感知是推的，能力是订阅的，判断同样是被事件触发注入的。三者都是“反应式”的，ReActive 才是一个完整的说法。

未验证：把假设当假设用

我们坚持把 Exp 标为未验证，这不是谦辞，而是设计约束。

经验天然具有三种失真。**虚假相关**：两件事一起出现过，不代表有因果关系；**分布漂移**：世界变了，旧倾向仍在；**自我实现**：如果系统总是按经验行事，它就很难收集到“不按经验会更好”的证据。任何一条都足以让“经验”在没有检验的情况下变成偏见。

因此 Exp 在系统中携带四样东西：

- **状态**：候选（candidate）、有支持（supported）、被反驳（contradicted）、已退役（retired）；
- **置信度**：由支持数与反驳数更新的一个标量，而非“是/否”；
- **来源**：指向产生它的 Mem 片段与 SKILL 调用记录，可回溯、可审计；
- **时间**：最后被验证的时间，用于衰减。

注入时的措辞同样是设计的一部分：经验以“通常”“上次类似情境里”这类倾向性表述进入提示词，而不是以事实陈述进入；并且明确告诉模型它可以不采纳。一个被反复验证的经验会逐步获得更高的置信度，但不会因此变成不可质疑的规则——只有被提升为 SKILL 的经验才具备“规则”的地位，而 SKILL 是可执行、可测试的，错误的 SKILL 会在测试里失败，错误的 Exp 只会在某一天悄悄出错。

失败模式与护栏

经验层的失败模式与对应护栏。“护栏”列是可以直接写进实现里的约束。

失败模式	表现	护栏
错误固化	一条错误经验被反复注入、反复“证实”	反例立即降级；支持数达到阈值才允许提升为 SKILL，且必须通过可执行检查
分布漂移	环境已变，旧倾向仍在生效	置信度随时间衰减；长期未被验证的经验自动退役
注入污染	经验文本覆盖了系统提示或安全策略	Exp 只能以“引用块”形式注入，不得指令化；注入内容可整体撤销
隐私外溢	经验中夹带用户数据、凭据、内部链接	整理阶段做脱敏与抽象；来源指针留在本地，不进入提示词

表中最后一条值得单独说：经验的“可复用”与“可泄露”是同一件事的两面。一条“这个仓库的发布流程总是先改版本号”是有用的经验，也是内部信息。因此我们的做法是：经验本身抽象到条件层，具体对象以指针形式保留在本地存储里，注入时只给条件与结论，不给原始材料。

落在双层架构的哪里

最后交代这一层在两个位置上的落点，否则它只是一个漂亮的概念。

Router 层用它做判别：这条消息要不要管、属于哪类任务、该给谁。这类判断高频、轻量、重复，正是经验最划算的地方——它把 Router 从“每轮重新理解世界”变成“认出这是哪一类情形”。

Worker 层用它做偏好：先看哪里、按什么顺序验证、用什么风格的输出。这类判断低频但具体，经验的作用是让执行带上“上次这么做成了”的先验，而不是让每个 Worker 都从零探索。

两层都不把经验当命令：Router 在经验与当轮证据冲突时以证据为准，Worker 在经验与测试结果冲突时以测试为准。经验的地位是“先验”，不是“结论”——这与上文“未验证”一节的约束是同一件事的两种说法。

3. 事件驱动

要让 Super-agent 具备**主动**性，首先需要改变系统的驱动方式。驱动 Agent 的主要输入不再是人类在 Session 中逐条键入的提示词，而是环境中的半结构化事件。在 Super-agent 中，群聊消息、邮件与日程提醒、代码审查评论、CI 运行结果等环境事件被称为 **Trigger**。通过监听并响应 Trigger，Agent 不必再像单 Session 模式下那样，等人类下达指令才被动执行一次。

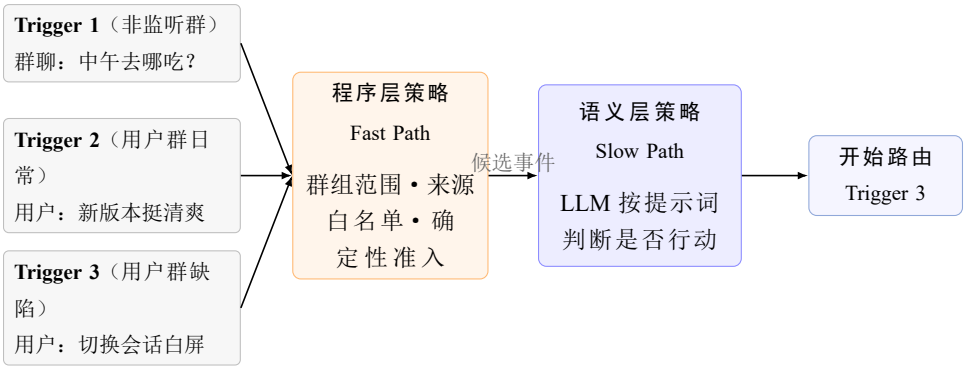
围绕一条环境事件从进入到处理的生命周期，下文将依次讨论四个递进的问题：如何通过程序与模型结合的**两级策略**过滤大量噪音（3.1 节）、如何通过**订阅与直通**机制将事件路由给 Worker（3.2 节）、如何利用 **Inline & Spill 与链式召回**向模型高密度地注入与按需召回上下文（3.3 节），以及如何利用**投机执行**根据环境信号在后台提前行动（3.4 节）。

3.1 事件过滤

在接入真实的群聊与代码平台后，环境事件流呈现出显著的“高噪音、低触发率”特征，绝大多数日常消息并不需要 Agent 介入。如果对每个事件都调用大模型从头判断，延迟和算力成本都会难以承受。借鉴计算机系统中的 **Fast Path / Slow Path** 思想，大模型推理可以看作一条灵活但高成本的 Slow Path，而将已经明确且稳定的高频判断固化为确定性代码，则构成了低延迟、低成本的 Fast Path。

因此，事件进入系统后的第一步是过滤，目标是在调用大模型之前先用确定性策略筛除噪音，只将需要语义理解的候选事件交给模型判断。我们采用程序策略与大模型相结合的**两级过滤**机制：

- **程序层策略（作为 Fast Path）**：针对事件来源、群组范围、用户或机器人白名单等确定性条件，直接通过代码进行快速准入检查，例如对不同来源分别设置“仅响应明确请求”“作为候选事件”或“直接忽略”，在调用模型前滤除大部分无关消息。
- **语义层策略（作为 Slow Path）**：通过程序层准入的候选事件交给 LLM，由它根据用提示词编写的策略和上下文判断是否需要行动。语义层同时作为面向人类的自然语言扩展点，方便随时通过提示词开关或新增策略，如下图所示：



两级过滤：先由程序层策略（Fast Path）筛除大部分无关消息，只把候选事件交给语义层策略（Slow Path）；左侧三条 Trigger 来自同一条 MSP 总线，来源不同而入口相同。

3.2 事件路由

第 2 节引入的全局 Router 层负责维持跨信道的连续性，但如果已分配任务的每个后续事件仍需经由 Router 层转发，这一全局节点依然会承受大量负载，推高系统的响应延迟。因此，Fast / Slow Path 同样适用于过滤后的事件路由。全局 Router 层仅负责新任务的冷启动与跨任务协调，而已确定归属关系的后续事件由程序直接发往对应的 Worker 会话，无需经由 Router 层转发。

在实际工作流中，同一个 GitHub Pull Request、同一个 IM 话题或同一封往来邮件串下的连续事件构成一条事件流，即 **Thread**：



Thread 示例：同一个 IM 话题、同一个 Pull Request，或同一次下载下的连续 Trigger，各自构成一条 Thread——结构相同，来源不同；谁订阅了这条 Thread，谁就收到它的后续事件。

同一个结构在个人场景里同样成立。一次文件下载就是一条 Thread：请求发出、进度推进、失败重试、最终完成，几个 Trigger 都挂在同一个对象上；一条天气变化也是一条 Thread：预警发布、等级升级、最终解除，每一条都是对同一件事的后续更新；一封会议邀约的往来（发出、改期、确认、临近提醒）同样是一条 Thread。

这类场景透露了一件我们起初没有预料到的事：个人助理需要的事件，绝大多数不是“请求”，而是“进展”。人不会把“下载完了吗”“降温了吗”做成一次次提问，而是希望助理一直关注这件事，等它有下文时再说一句。Thread 正是这种“关注”的形式——助理不必为“下载”或“天气”写专门的业务逻辑，它只需要订阅这两条 Thread，然后在任意一个 Trigger 到达时决定要不要介入。

个人场景：一次下载、一条天气预警，两条 Thread 同时推进

全程无人下指令；助理只在“有下文”时说一句

信号	`file.download.started` 4.2 GB，已排队，前台不受影响
信号	`weather.alert.upgraded` 18:00 起有雨，风力 6 级
助理	18 点有雨，出门记得带伞。（一句话，不追问）
信号	`file.download.done` 校验通过，已解压到 `~/data`
助理	数据集就绪：12,480 个文件。（交付，不再解释过程）

个人场景下的两条 Thread：下载与天气各成一条，互不阻塞；助理不追问、不汇报过程，只在“有下文”时插一句。这两条 Thread 的形状与原图的 IM、Pull Request 完全一样——这正是第 3 节所说的“成串的事件比事件的来源更稳定”。

在 Router 层根据某个 Thread 的初始事件创建 Worker Session 后，该 Worker 会直接订阅对应的 Thread，从而将事件路由分为如下图所示的两条路径：

- **已订阅（Fast Path）**：同一 Thread 下的后续事件由程序直通已订阅的 Worker，无需 Router 层再次调用工具转发。若新事件超出了当前任务的范围，Worker 也可取消订阅或交回 Router 层重新分派。
- **未订阅（Slow Path）**：Router 层仍可观测所有事件以掌握全局状态，但仅负责判断和分派尚未被订阅的新事件。



订阅与直通：初始事件经由 Router 层创建 Worker 并建立订阅，同一 Thread 的后续事件直接发往该 Worker

此外，Thread 与 Worker 之间的订阅是多对多的关系。一个 Worker 可以同时订阅多个相关的 Thread，同一个 Thread 也可以被多个 Worker 订阅。当多个 Worker 在同一个工作环境或讨论中发生交集时，该 Thread 下的新事件会同时直通给所有订阅它的 Worker，使它们无需点对点通信，也有机会获知彼此的操作并自行解决冲突。而在输出侧，Worker 仍将关键结论汇报给 Router 层，由 Router 层统一向用户回复并同步全局记忆。

3.3 上下文组织

在事件经过过滤与路由、进入目标 Session 后，下一个问题是应当向模型的上下文窗口注入多少信息。由于系统输入从有限的人类提示词转向了环境持续产生的半结构化事件，工具调用与事件注入的结构设计变得更加重要。注入过多会挤占窗口、稀释模型注意力，注入过少又会迫使模型反复调用工具读取，增加执行步数。浪费上下文窗口，就是浪费模型的智能。

为此，我们可以将有限的上下文窗口当作 Cache 来管理。借用编译优化与计算机系统中的概念，上下文注入的设计可以归纳为两个基本操作。**Inline** 是指将近期很可能用到的信息直接内联进当前上下文窗口，以省去额外的工具调用往返。**Spill** 则是指将超出窗口预算或低频使用的完整信息留在上下文窗口之外，仅在窗口内保留引用或摘要，供模型按需读取。

结合计算机系统设计中经典的**局部性（Locality）**思想，我们可以用 **Inline & Spill** 对事件注入和工具调用逐步优化：

1. **从最少信息开始：**最初只提供当前对象的 ID 和内容摘要。
2. **利用局部性逐步 Inline 关联信息：**将近期很可能用到的信息 Inline 到上下文中，先附上紧密关联对象的 ID，再附带时空相邻的内容，例如内容前 50 字、IM 消息的前后相邻消息与所回复的消息，或代码评论所在评审讨论中的相邻评论。
3. **超出上限则 Spill：**对无法全部 Inline 的长内容设定较小的长度上限，将超出部分 Spill 到窗口之外。一种简单直接的做法是截断超出部分并保留续读入口，模型需要完整内容时再调用工具读取。

此外，在大量后续事件直通 Worker、且系统在长期运行中累积了许多历史任务后，Router 层既不能把所有任务的全量历史都放入自己的上下文窗口，又需要在用户跨信道提起某个旧任务时找回上下文。将 Inline & Spill 一致地应用到系统中的各类对象后，就得到了一种按需展开的多跳检索机制，即**链式召回（Chain of Recall）**。这种链式召回过程缓解了 Router 层在长程中丢失先前信息的问题，如下图所示：

具体来说，系统为 Trigger、Thread 和 Worker Session 等对象赋予唯一 ID，并在注入时内联紧密关联对象的 ID，使这些对象在底层构成一张引用图。由于每一次调用工具召回的对象本身

同样遵循 Inline & Spill，其返回结果中又会内联下一跳关联对象的摘要与 ID，模型便可以从当前事件出发，顺着引用关系逐步按需召回上下文。

例如从一条当前消息读取所在 Thread，再从 Thread 内联的 ID 追踪到相关的 Worker Session，进而找到该 Worker 关联的另一个代码评审 Thread。

3.4 提前行动

借鉴计算机系统中的**投机执行（Speculative Execution）**思想，Agent 可以在还不确定结果是否会被采用时，根据环境信号在后台提前尝试，通过消耗额外的后台算力来减少人类的等待与介入成本。结果有价值就交付，没有价值就丢弃。

在个人助手场景下，当收到一封会议邀约邮件或待办消息时，Agent 可以先在后台核对日程冲突、搜集相关背景资料并备好回复草稿，待用户过目后即可直接确认发出。在研发协作场景下同样如此，例如在 DeepSeek Harness 的内部用户群中，就有一条“静默复现并修复”策略：当群里有人反馈问题或讨论改动方案时，Agent 先在后台尝试复现并寻找修复方案，只有在确认问题并找到简单的修复方案后，才会把结果交付给用户，否则全程保持安静。



链式召回：每次召回的对象均内联下一跳关联对象的 ID，支持沿引用关系逐跳展开上下文



投机执行示例：群内还在讨论解决方案时，Super-agent 已在后台提交了修复 Pull Request

3.5 事件从哪里来：节点、信号与能力清单

3.1 至 3.4 节讨论的是一条事件进入系统之后如何被过滤、路由、组织与提前处理。还剩一个问题没有回答：这些事件、这些能力，究竟从哪里来、怎么接进来。这就是本节要补的第二个补丁：让“反应”通用。

接入的代价：N×M

回到前面的 Trigger：Super-agent 以它作为主要输入，环境事件经由过滤与路由进入对应的 Worker。这套设计在语义上是完备的——任何事件都可以被描述、被过滤、被分派。但在实现上，它把“事件从哪来”留给了工程：群聊要接一个机器人，GitHub 要接一个 Webhook，CI 要接一个轮询器，某台机器上的文件变化要接一个 watcher，而个人场景里的“下载完成”“天气预警升级”又要各写一个连接器。每多一个来源，就多一个连接器。

能力侧同样是这个问题。工具要注册进工具表，远端 Agent 要按 A2A 对接，MCP 服务要起一个子进程做桥接，设备上的命令执行要装 daemon 并约定通道。这些差异最终被一层“适配层”吸收，而适配层不属于任何协议：它无法被复用，无法被推理，也无法被别的实现继承。

于是系统呈现出一种不对称：事件有统一的语义（Trigger），却没有统一的入口；能力有统一的调用方式（工具调用），却没有统一的接入方式。当来源从三种变成三十种，适配层的厚度就成为系统通用性的上限。

这正是 N×M 问题：N 种能力、M 种接入方式，需要 N×M 份适配。现有的协议各自解决了其中一片——MCP 解决了“模型与工具”之间的客户端–服务端调用，A2A 解决了“Agent 与 Agent”之间的任务协作——但环境事件（一条文件变化、一个传感器读数、一次进程退出）在两者中都没有位置：它们既不是被调用的工具，也不是被委托的任务，而是自己发生的。

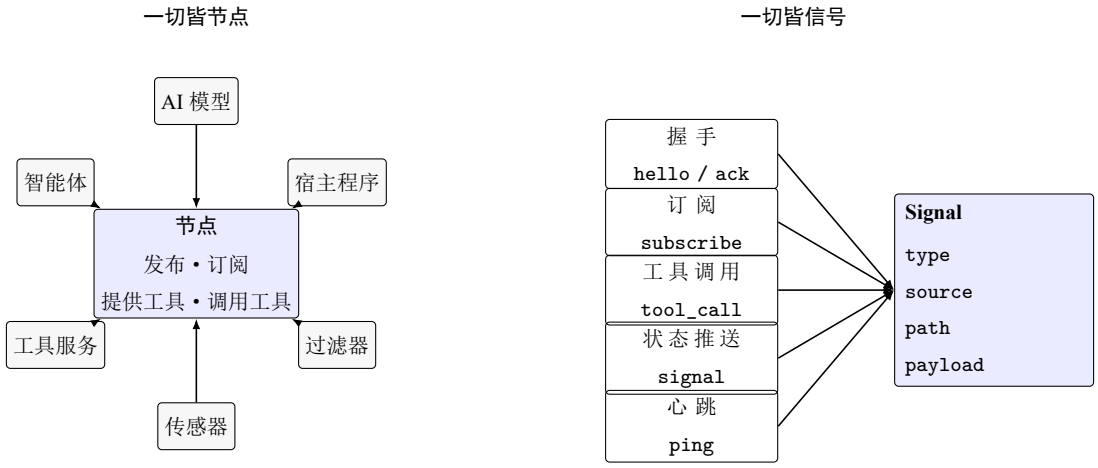
节点与信号：只留两个概念

MSP（Model Sensing Protocol）的做法是把概念收到只剩两个。

节点（Node）是通讯的参与方：工具、模型、传感器、宿主程序、远端服务都是节点，各有全局唯一标识。节点不预设“客户端/服务端”角色——同一个节点可以发布信号、订阅信号、提供工具、调用工具。

信号（Signal）是通讯的最小数据原子：握手、订阅、工具调用、状态推送、心跳、错误，全部编码为同一种结构，仅以一个 `type` 字段区分。信号的 `path` 是它的逻辑地址，支持单段通配 `*` 与多段通配 `**`。路径命名就是这套协议对世界的“分类法”：研发场景里它是 `notify.web.download`，个人场景里它是 `file.download.done`、`weather.alert.upgraded`、`calendar.invite.changed`。它们形式完全一样，接进来之后所走的过滤、路由与分派也是同一套。

这两个概念带来一个直接后果：一切皆节点，一切皆信号（下图）。模型不再是“调工具的人”，传感器也不再是“被轮询的东西”——它们都是节点，用同一种方式说话。



节点只有一个接口（发布 / 订阅 / 提供工具 / 调用工具），信号只有一个结构（以 `type` 区分）。MSP 的全部通用性来自这两条约束。

把差异压到两个概念上，换来的是三件事：新能力只需注册一个节点，新事件只需约定一条路径，新设备只需在该设备上起一个节点。Router 侧不需要知道对方的实现语言、部署位置与厂商。

握手时就说清楚：能力清单

节点之间的第一次对话是握手，交换的是各自的能力清单（Manifest）。清单声明四件事：本节点会发布哪些信号路径、提供哪些工具（含输入输出的 Schema）、可订阅哪些上下文资源、启用哪些通讯模式。

把“启用哪些通讯模式”放进清单，是 MSP 与常见协议不同的地方：两个节点在建立连接之前就确认了“我们可以用哪种方式交流”，而不是连上之后再试探。这为模式降级提供了依据——一端的模式对方不支持时，按清单里声明的降级目标切换到更低阶的模式，而不是直接失败。

清单还是“能力发现”与“权限”的共同载体：节点看到的是什么，取决于它被允许看到什么；一条 `manifest_update` 信号可以在不重连的情况下增量地更新能力视图。

核心信号类型。全部交互都落在这张表里，type 之外没有第二种数据原子。

类型	方向	含义
hello / ack	双向	握手，交换能力清单
subscribe / unsubscribe	双向	订阅、退订某条路径（可为通配模式）
subscribe_ack	双向	订阅确认
signal	单向	发布一条数据信号
tool_call / tool_result	请求/响应	同步工具调用
manifest_update	单向	能力清单增量推送
ping / pong	单向	心跳探活
error	单向	错误通告

五层，与三个正交的扩展轴

MSP 的实现组织为五层：发现与宣告、传输、协议、模式、应用。层间以窄接口相连——协议层只依赖传输层的“收发字节”，不感知发现机制，也不感知具体传输协议。

真正的收益不在分层本身，而在于三个扩展轴彼此正交：换一种发现机制（共享目录、注册中心、DHT）不改协议层；加一种传输后端（TCP、WebSocket、QUIC）不改协议层；加一种通讯模式同样不改协议层。

五层架构与可替换的实现。标注“已实现”的部分构成第 7.3 节所说的“已完成”。

层	职责与当前实现
应用层	统一入口 MSPNode：声明工具与信号源、发布、订阅、调用、注册模式（已实现）
模式层	可插拔的通讯模式：请求–响应、发布–订阅为核心；多轮会话已作为第一个扩展模式实现；流式、任务生命周期留待扩展
协议层	握手、订阅表、按路径路由、基于 reqid 的并发调用关联（已实现）
传输层	字节收发与分帧：TCP（长度前缀分帧）与 WebSocket（二进制帧）已实现，QUIC 留待扩展
发现/宣告层	一对正交接口：“怎么找到别人”与“别人怎么找到我”；共享目录扫描与手动指定已实现，DHT 留待扩展

一种信号做所有对话

模式层是这套设计里最容易被低估的部分。常见的协议把“请求–响应”写进内核，于是“发布–订阅”“多轮往返”“流式”只能以补丁的形式外加，彼此语义不通。MSP 的做法是：协议层只认 pattern 字段，据此查表把信号交给对应的模式处理器；模式处理器是可插拔的独立模块，实现自己的状态机。

结果是“加一种交流方式”不需要动协议核心，也不需要让已有的模式知道新伙伴的存在。多轮会话模式就是这样接进来的：它在工具调用之上叠加了“还需要一轮输入”的状态，而协议层对此一无所知。

与模式层配合的是通配订阅。没有通配，订阅就是一次枚举——你得先知道世界上有哪些路径；有了通配，“订阅 `system.**`（其下任意层级）”这样的表达才成立，节点才能对自己还没见过的对象保持敏感。这也是“反应式”在实际系统中能成立的前提：助理不可能预先枚举它将来会关心的一切。

MCP 与 A2A 是特例，而不是对手

把 MSP 与现有协议对照，结论比“谁更好”更简单：MCP 是“请求–响应模式 + 客户端/服务端拓扑”的特例，A2A 是“节点与节点委托任务”的特例。MCP 的能力握手可以一一映射到 Manifest，`tools/call` 映射到 `tool_call/tool_result`，`notifications` 映射到单向 `signal`；反过来，发布–订阅在 MCP 中没有一等公民的对应物——这不是 MCP 的缺陷，而是它把拓扑写进了内核的必然结果。

因此我们的判断是：MCP 与 A2A 是同一抽象在不同拓扑下的投影，而 MSP 是那个抽象本身。它不要求生态迁移，只要求生态有一个可以映射到的公共底座。

落回 ReAct：观察可以来自订阅

最后回到本文的主线。MSP 对 ReAct 的直接影响只有一句话：观察的来源扩展了。

在经典 ReAct 里，Observation 只来自工具返回值——模型必须主动伸手，才能知道世界上发生了什么。于是“等待”只能被伪装成“行动”：调一个 `sleep(45)`，烧掉一整轮思考–行动–观察，醒来再看一眼。而在 MSP 之上，等待可以写成订阅：模型订阅一条路径，然后继续做别的事，等到状态真正变化时被唤醒。感知的责任从模型下放到协议层，等待从“行动”降格为“订阅”。

这就是前文 Trigger 在协议层的落点：Trigger 不是一个需要逐个适配的外部事件，而是信号流上的一条路径。也正因如此，同一个机制同时给出了通用性：任何能力接进来都是一个节点，任何事件接进来都是一条路径，而 Router 与 Worker 的代码不需要为此改动。

通用性怎么度量

我们不打算给“通用”下一个含糊的定义，而是给出三个可检查的判据：

- **新增能力**：注册一个节点、声明它的工具与信号路径即可，Router 侧零改动；
- **新增事件源**：约定一条路径即可，不需要新的连接器类型；
- **新增交流方式**：实现一个模式处理器并注册，协议核心零改动。

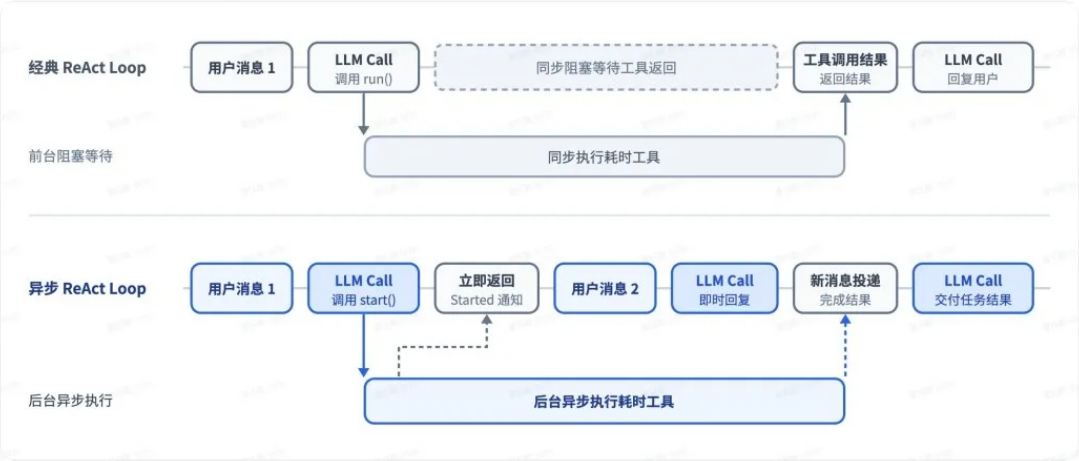
这三条都能在现有实现上直接验证，7.3 节给出了对应的证据。需要说明的是，通用性并不等于“什么都能做得好”：MSP 只规定了“怎么接进来”，不规定“接进来之后怎么判断”——后者是 2.3 节回答的问题。

4. 面向人的交互

第 2 节的双层架构从上下文层面保证了跨信道的**连续**，但要在实际的人机交互体验中同时做到**连续**与**高效**，还需要在交互方式与交互渠道上解决两个问题：一是如何通过**异步 ReAct Loop** 消除后台耗时操作对前台对话的阻塞，使会话无需等待上一个命令或任务完成即可随时接受新指令（4.1 节）；二是当交互发生在用户熟悉的不同工作信道时，如何让系统的输出方式匹配对应信道的带宽与人的使用习惯（4.2 节）。

4.1 全双工交互

在经典的 ReAct Loop 中，模型发起一次同步的 `run()` 工具调用后，循环会阻塞，直到工具执行完毕并返回最终结果，在此期间会话无法响应任何新的用户输入。要消除这一阻塞，需要将对话从一问一答的半双工模式转变为全双工模式，如下图所示：



经典 ReAct Loop 与异步 ReAct Loop 对比：异步调用立即返回 Started 通知，后台工具运行期间仍可响应新消息

为此，我们将耗时的命令与子任务工具调用改为异步的 `start()` 和 `stop()`，构成**异步 ReAct Loop**。模型调用 `start()` 后立即收到“已启动（Started）”的返回，循环不再等待该工具；在后台工具执行完成后，系统再将最终结果作为一条新的事件消息投递进会话；同时，模型也可以根据上下文调用 `stop()` 主动终止执行时间过长的后台命令。

由此，前台对话响应与后台任务执行得以解耦，Agent 在后台运行耗时任务的同时也能响应人类输入。新输入在当前推理步结束后即被处理，不必等待后台工具完成。

这一异步任务机制已经在 DeepSeek Harness 中实现，但尚未默认开启。

4.2 多信道交互

不同工作信道具有不同的信息密度与交互习惯，系统的输出需要与之匹配。如果将面向单人终端的交互方式直接搬入群聊，把冗长的推理过程与密集的工具调用原样输出，在多人协作中极易造成信息过载。

在 Super-agent 中，双层架构已经解决了其中一部分问题：具体任务中冗长的推理过程和密集的工具调用都被隔离在 Worker Session 内部，不会直接输出到外部信道，而由 Router 层统一负责对外交互。在此基础上，系统还需要接入用户日常使用的各类工作环境，并通过提示词针对不同信道定制交互规范：

- **IM 等即时通讯信道：**侧重低打扰的轻量沟通，像人类同事一样只在需要决策或验收时发送通知，只问直接问题、给简短结论。我们验证过的一条简单有效的硬性规定是单次回复不超过 100 词。
- **GitHub、邮件与文档等异步信道：**侧重结构化、高信息密度的异步交付，例如在创建或更新 Pull Request 时按项目规范撰写完整的问题背景、改动说明与验证结果，或在个人助手场景下直接起草结构化的邮件与文档摘要。
- **日历、系统通知与设备端等窄带信道：**侧重恰好在需要的那一刻出现一次。日历项只写“何时、何地、要带什么”，不写过程；通知一条一句话，只在需要用户动手时才发；手表、音箱这类设备端只念结论，不念推理。带宽越窄，判断“该不该发”就比“发得多好”越重要。

值得多说一句的是，这三类规范并不是三份写死的代码。在 MSP 里，一个信道就是一个节点，它的输出规范写在节点自己的能力清单里（3.5 节）：Router 分派任务时，Worker 拿到的是“往哪个节点、按什么规范说话”，而不是一段散落在提示词里的风格要求。于是新增一个信道——比如一块新手表、一个新 IM——只需注册一个节点，不必把提示词再抄一遍。

再进一步，同一个人在同一个信道上的偏好，本身就是可以沉淀的经验。有人在 IM 上要“结论先行”，在邮件里要“背景齐全”，在通知里只接受“需要他动手”的那一类；这些偏好不必每次重述，它们会在后台被整理成经验（Exp），在该信道的事件触发时注入（2.3 节）。所以信道规范其实有两层：一层写在协议里，由节点声明；一层写在经验里，属于这个人。

5. 计算、状态与环境的分离

现有的本地 Agent 默认将推进 Agent Loop 的**计算**、需要持久保存的会话**状态**，以及代码仓库与本地工具链等执行**环境**绑定在同一台设备上。一旦设备休眠或离线，Agent 就会停止运行，且无法访问其他设备上的环境。要让 Super-agent 做到**持久**在线并灵活接入各类真实工作环境，我们可以借鉴计算机系统中**存算分离（Compute-Storage Separation）**的思想，分两步解耦这三者的绑定：第一步先将推进 Agent Loop 的计算与持久化的会话状态分离，使 Agent 摆脱单机生命周期限制、实现云端持久在线（5.1 节）；第二步再将云端的 Agent Loop 与重状态的执行环境解耦，让真实工作环境留在原地并支持跨设备访问（5.2 节）。

5.1 计算与状态分离

推进 Agent Loop 的计算可以做成无状态的，需要持久保存的是会话状态。因此，第一层解耦是将推进 Agent Loop 的计算放入云端无状态的弹性计算容器中，而将 Session 等持久化状态存入数据库。无状态计算容器可以水平扩展、重启或替换，会话与任务的连续性则由持久化存储保障，从而使 Agent 摆脱单台本地设备的生命周期限制，实现云端持久在线。

5.2 计算与执行环境分离

在 Agent Loop 与会话状态常驻云端后，进一步的问题是如何访问任务所需的执行环境。模型工作所需的上下文很大一部分要通过工具调用从代码仓库、本地文件、依赖服务等执行环境中获取，从这个意义上说，广义的上下文也包含了执行环境本身。

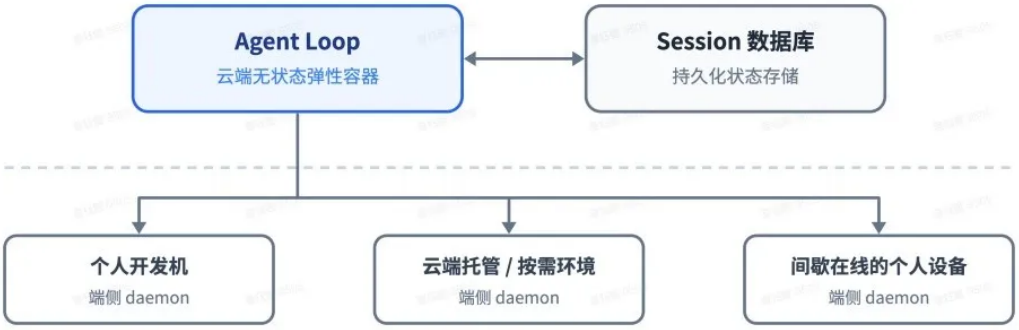
围绕这一问题，常见的思路有两种：一种是在云端启动沙箱、拉取代码并尝试重建和同步环境，但由于真实工作环境往往包含未提交的本地修改、私有凭据、内网服务依赖以及特定的硬件或工具链，在技术上同步这样的环境不仅成本很高，还容易出现信息遗漏；另一种是直接要求用户放弃本地环境、迁往云端工作，但对很多用户来说迁移成本过高。

既然难以复制和同步的是环境本身，就不必让执行环境跟着 Agent Loop 上云。第二层解耦是让 Agent Loop 在云端运行，而让重状态的执行环境留在原地，仅在端侧部署一个不保存会话状态的轻量 daemon，它只负责执行命令与返回结果，相当于把执行环境留在云端之外，由 Agent 按需读取其中的上下文。

这一思路与 Anthropic 提出的“脑手分离”（Decoupling the brain from the hands）一致，同样让 harness 保持无状态，把会话日志存放在 harness 之外，并通过统一接口按需接入执行环境。两者的区别在于场景。那篇文章描述的是托管服务，主要讨论按需创建的沙箱和客户 VPC 内的资源；这里首先要接入的是用户日常使用、可能间歇在线的个人机器，上面有未提交的修改和

本地凭据。因此，这种设计既支持 **BYOM（Bring Your Own Machine）**，即把用户日常使用的开发机或个人电脑接入云端 Agent，也支持接入云端托管机器或按需创建的计算环境。

由此，一个 Agent 不再受限于单台机器，而是可以在不同的执行设备之间切换，根据任务需要选择环境来执行命令并获取环境中的上下文。它甚至能够支持间歇在线的设备，例如在用户的某台个人设备重新上线后，再由云端 Agent 自动触发并完成需要在该设备上执行的任务。下图展示了计算、持久化状态与各类执行环境分两层解耦后的结构：



存算分离与执行环境解耦：**Agent Loop** 与持久化状态在云端分离，并通过端侧 **daemon** 按需接入各类执行环境

6. Watchdog

要实现**持久**在线，除了第 5 节的存算分离，还需要让系统在长期运行中能从异常中自动恢复。持久在线的分布式系统很难仅靠主系统内部逻辑保证长期可靠运行，需要在主系统之外引入独立的外部控制回路，即 **Control Loop**。

同时，在 Agent 系统中，仅靠传统的进程存活探测或监控脚本是不够的，因为许多异常并非进程崩溃，而是模型在执行中陷入无效循环或偏离目标。因此，长期运行的 Agent 系统不仅需要监控基础设施的存活，还需要监控任务本身的进展。

我们在主系统外部配置了一个独立的 **Watchdog Session**，用定时消息驱动它，构成外部的 Control Loop。Watchdog 除了处理接口报错、状态恢复失败等基础设施故障，还会由模型判断各个 Worker 是否有实质进展、方向是否偏离整体目标。发现异常时及时介入修复，遇到外部偶发故障则持续复查直至恢复正常。

由于 Watchdog 本身也是一个具备完整工具能力的 Agent Session，维护者可以用自然语言向它询问故障原因，也可以让它在线修正主系统的行为，例如增删提示词、调整触发规则。

7. 初步实验

为了评估前文的核心调度与路由设计，我们从两个层面进行了初步实验：首先通过端到端实验对比双层架构、订阅直通与单 Session 方案（7.1 节），随后单独测试 Router 层的批处理（7.2 节）。由于实验基于合成负载与简化环境，部分设置还可以更加严谨，以下结果仅作为初步参考。

7.1 消融实验

我们用群聊和 GitHub CI 消息流同时驱动四种端到端方案：单次实验包含约 300 条消息，其中约 20 条是工作请求，其余是闲聊、跟进、叫停和 CI 消息。编码工作用一个只消耗时间的工具代替，因此实验衡量的是系统的调度与协作能力，而非对单个任务的执行能力。每种方案重复运行 5 次。

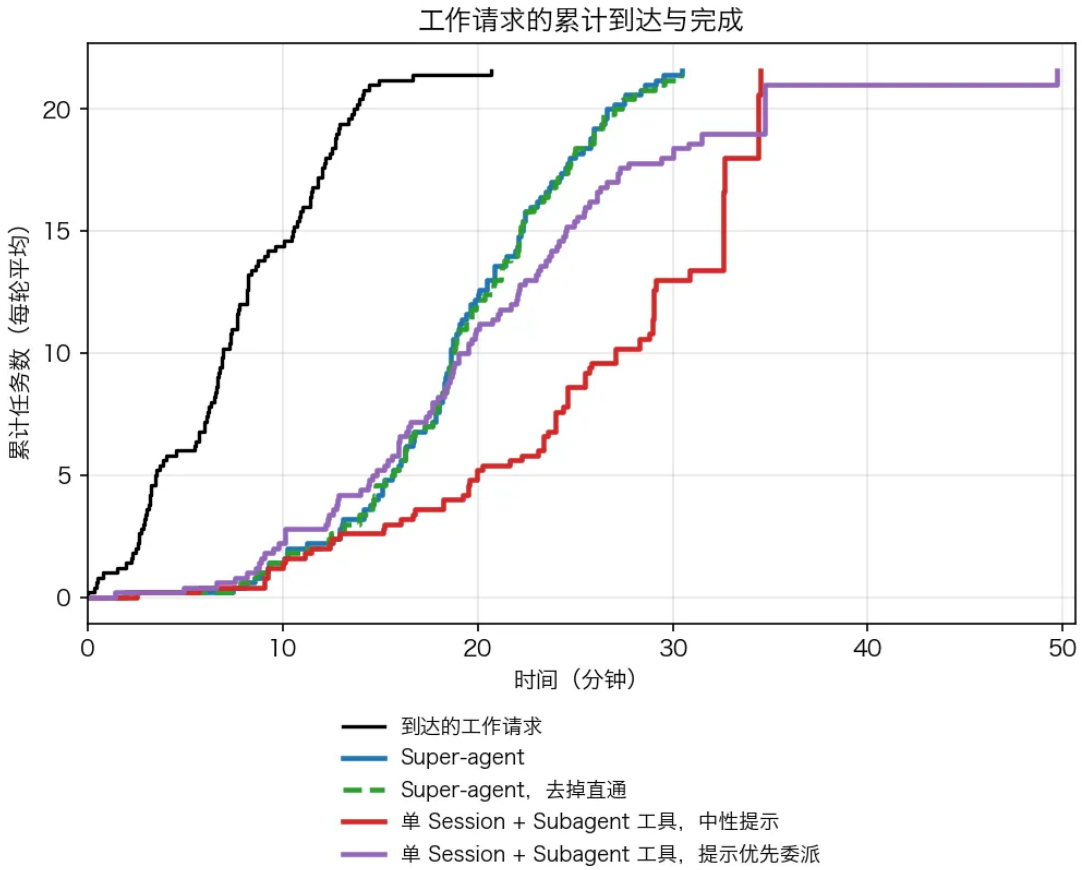
对比的四种方案包括 **Super-agent**、**去掉直通的 Super-agent**，以及两种提示词配置下的**单 Session + Subagent 工具**。

其中，**中性提示**写为“*Use a subagent for a piece of work when appropriate...*”

提示优先委派则写为“*Every message of the chat reaches you, so work you run yourself keeps stopping for new messages. Prefer handing each piece of work to a subagent of its own, and do only small things yourself...*”。

从任务完成时间看，Super-agent 的 p50 与 p95 分别为 12.4 分钟与 15.7 分钟，显著优于带 Subagent 工具的单 Session。原因在于，带 Subagent 的单 Session 在实验场景下受提示词影响较大，主 Session 常常决定自己直接执行任务，导致总完成时间长、性能不稳定。在中性提示下，模型在 5 次运行中一次都没有调用 Subagent，所有工作都在主 Session 中进行并不断被新消息打断，p95 达到 29.5 分钟；改为提示优先委派后，模型在 5 次运行中有 4 次进行了委派，完成时间有所缩短但整体仍慢于 Super-agent，且剩下 1 次它判断工作“很小”而选择自己执行，形成了图中的紫色长尾。相比于在单 Session 中依赖提示词工程和模型临场判断分不分工，双层架构通过让 Router 不直接执行具体工作、Worker 仅接收所订阅 Thread 的消息，在系统层面上缓解了这一问题。

从系统开销看，由于维护了多个独立会话，Super-agent 单次实验的读缓存 token 高于单 Session，是以更多的 token 消耗换取更短且更稳定的完成时间。而在 Super-agent 内部去掉直通机制后，任务完成时间与完整版基本一致，但单次实验中 Router 需要做出的决定增加了约 30%，总读缓存 token 从 2323 万增加到 3177 万，增幅约 37%。这表明在实验负载下，直通机制不提高任务完成质量与速度，但为 Router Session 节省了可观的 token 与推理开销。



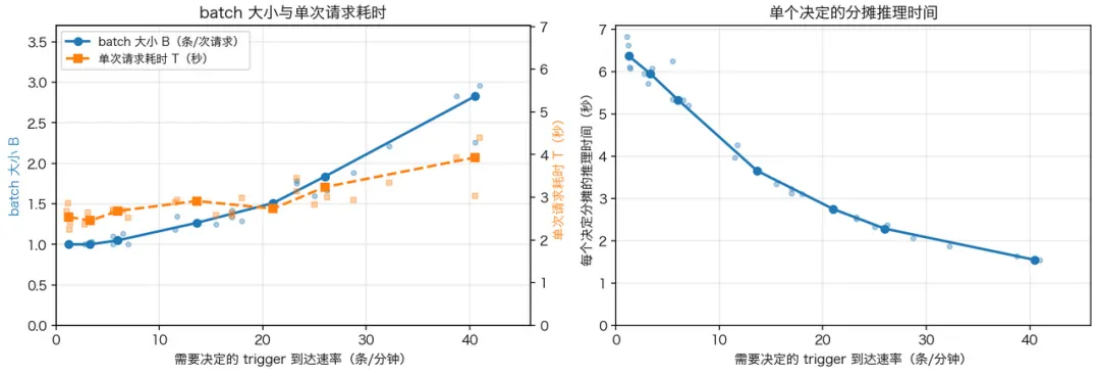
工作请求的累计到达曲线与四种方案的累计完成曲线（5 次实验运行均值；消息间隔与工作时长按 1/4 缩放，模型调用耗时为真实值）

方　案	任务完成时间 P50 / P95（分钟）	单次实验读缓存 Token（全部 Session）
Super-Agent	12.4 / 15.7	2323 万
Super-Agent，去掉直通	12.6 / 15.5	3177 万
单 Session + Subagent 工具，中性提示	20.4 / 29.5	605 万
单 Session + Subagent 工具，提示优先委派	13.8 / 29.0	1082 万

注：每种方案重复运行 5 次；任务完成时间指从工作请求到结果消息的耗时。

7.2 Router 层的批处理

为了验证 2.1 节中单个常驻 Router Session 的吞吐上限与批处理机制，我们单独对 Router 层进行了压力测试：让 Router 调用真实模型、Worker 使用 Mock 模型，并将需要决定的 Trigger 到达速率从每分钟约 1 条逐步提升至约 40 条，观察平均 batch 大小、单次请求耗时与单个决定分摊的推理时间。



不同 Trigger 到达速率下 Router Session 的 batch 大小、单次请求耗时与单个决定的分摊推理时间

如上图所示，当需要决定的 Trigger 到达速率从每分钟约 1 条升至约 40 条时，平均 batch 大小从 1.0 升至 2.8，而单次请求耗时仅从 2.5 秒增至 3.9 秒，增长慢于 batch 大小，从而使单个决定分摊的推理时间从 6.4 秒降至 1.6 秒。这意味着当外部事件突发拥堵时，Router 层能够通过自动合并积压事件提高有效吞吐。这也解释了为什么在 7.1 节中，即使去掉直通使 Router 的峰值负载增加，仍未触及单 Router 的吞吐瓶颈。因此在我们目前的团队规模下，无需过早对 Router 层做水平切分而牺牲全局上下文的连续性。

7.3 实现状态与评测方案

本文的两个补丁成熟度并不相同。为了避免读者把“已经在跑”与“还在试”混为一谈，本节用一张表把两者分开，然后分别说明各自的证据强度。

实现状态。**已完成**指有实现、有端到端演示；**实验性**指机制已跑通、但尚无稳定性结论。

层	内容	状态	证据 / 缺口
MSP	五层架构、TCP/WebSocket 传输、核心模式与多轮模式、共享目录发现、能力清单握手、通配订阅、节点预设	已完成	三类端到端演示：跨节点工具调用、信号订阅一致性、多轮会话模式接入（见下文）
MSP	MCP 互操作（把 MCP 服务桥接为节点、把节点暴露为 MCP 服务）	已完成	与 MCP 的语义映射逐项对齐；反向映射不存在，即 MCP 是子集
Mem	情景与语义记录的采集、召回	实验性	采集与召回可运行；召回质量未评测
SKILL	声明式能力清单、引用完整性与前置条件校验	实验性	校验可运行；与经验层的联动仍是原型
Exp	后台整理、候选池、触发注入、结果回流	实验性	最小闭环可运行；**有效性未验证**，评测方案见下文

MSP：三条演示说明了什么

MSP 的验证是机制性的：它不回答“MSP 比 MCP 快多少”，只回答“这些机制是否真的成立”。

演示一：跨节点工具调用。两个节点通过共享目录零配置互发现，A 在能力清单里声明一个工具，B 发起调用：发现 → 惰性建连 → 发送 `tool_call` → A 执行 → 回送 `tool_result`。调用往返在毫秒量级，并发多个 `reqid` 时互不串扰——验证了并发关联机制。

演示二：信号订阅。一个“传感器”节点在 `sensor.temp` 上周期发布，两个消费者订阅该路径。三条结论被验证：订阅建立后每次发布都能送达（订阅一致性）；未订阅的节点收不到（路由隔离）；任一订阅者下线后，对端订阅表被正确清理，不影响其余订阅者。

演示三：多轮会话模式。加载第一个扩展模式 `msp/multi-round`，在工具调用之上叠加多轮状态。模式处理器在不修改协议核心的情况下被注册并参与路由——验证了“每一层皆可扩展”这一架构承诺，而不只是宣称。

这三条演示对应的正是 3.5 节给出的三个判据：新增能力、新增事件源、新增交流方式，都不需要改动核心。需要强调的是，它们演示的是接入的通用性，不是判断的质量。

ReActive 与轮询的对照

3.5 节说“等待可以写成订阅”。这句话是否成立，取决于订阅相对轮询是否真的改变了感知延迟与开销的结构。我们做过一次机制性的对照：让 Agent 依赖一个会被外部修改的对象做判断。

轮询组的做法是每个推理步前重读一次，或者调一个短睡眠再读；订阅组的做法是订阅该对象的路径，收到变更信号时由系统注入上下文。观察到的差别是结构性的而非数值性的：轮询组的感知延迟等于轮询周期，且与轮询频率成正比地消耗 token；订阅组的感知延迟由事件触发，与周期无关，正常路径上不产生额外读取。

我们没有在标准基准上比较这两者，也不认为这样的比较有意义——它衡量的是同一个能力在两种表达方式下的代价，而不是两种系统谁更强。这一点在下一节对经验层的态度上同样适用。

经验层：为什么现在还不能给结论

Exp 的最小闭环是可以运行的：交互产生记录、后台整理出候选、触发时注入、结果回流更新置信度。但这距离“证明它有用”还差三件事：

1. **没有稳定的评测集**。经验的价值体现在“同类情境的重复出现”上，构造这样的分布本身就是一项工作：需要真实的任务流，而不是合成的对话。
2. **没有分离混杂因素**。一个有经验的系统同时拥有更多上下文、更短的提示、不同的措辞；要判断是“经验”起作用，必须把这些因素逐一固定。
3. **没有失败率的统计**。经验最危险的失败是“错误固化”，而它的表现是长期的、稀疏的；短周期的实验看不见它。

因此我们给出的是评测方案，而不是结果：

经验层的评测方案（尚未执行）。前两项衡量收益，后两项衡量代价。

指标	定义	测量方式
习惯命中率	注入的 Exp 中被采纳、且事后被判定正确的比例	离线回放历史任务流，人工或模型判定
注入精度	注入的前 k 条中与情境相关的比例	与人工标注的相关性对比
错误固化率	一条被反驳的 Exp 在系统中存活的平均轮数	注入反例，观察降级链路是否触发
代价	单任务的时延与 token，相对“无经验”基线	固定其余因素后的 A/B

在拿到这些数字之前，本文对经验层的所有表述都应被读作*设计主张*，而不是*实验结论*。这与我们给系统的建议是同一件事：把经验当假设用。

一句总结

MSP 是通用型实现的关键——它把“接进来”从工程适配变成协议约定，并且已经在跑；Exp 是习惯判断的关键——它把“判断”从每轮重算变成可注入的先验，但它是实验性的，而

且我们在设计上就承认它未验证。两句话合起来，才是本文标题的完整意思：ReActive 是个人助理所需的全部，但“全部”里有一部分还只是承诺。

8. 开放问题

第 2 至第 6 节介绍的五个模块构成了支撑 Super-agent 运转的基本架构，第 7 节的实验也验证了其中核心调度与路由机制在多任务并发下的效果。这套系统之所以能够以较低的复杂度运作，很大程度上是因为我们在当前阶段做出了一系列工程和场景简化。超出这些简化的适用范围后，在双层架构的扩展（8.1 节）、脱离 Session 后的人机协作（8.2 节）、上下文边界与 A2A（8.3 节），以及安全与隐私（8.4 节）上，仍有一些问题需要解决。

8.1 如何扩展双层架构

前文通过“单常驻 Router Session + 订阅直通 + Worker 间不直接通信”的简化组合，以较小的工程代价覆盖了我们目前的大多数日常任务。但当任务规模扩大、耦合度提高，超出单个 Router Session 的承载范围时，具体的层内结构并无定论，Router 层何时需要水平切分、粒度如何划分，以及同层 Worker 之间何时需要引入直接通信、采用何种协作拓扑，仍有待在更多场景中检验。同时，系统与模型的协同设计（Co-design）也是一个值得探索的方向，针对 Router 层的事件判别、批量分派与上下文召回等任务进行专项训练，有可能进一步提升 Router 层的吞吐与决策性能。

与此同时，前文中常驻的 Router 层配合基于引用图的链式召回，主要解决的是近期任务与会话历史的按需检索。当运行周期拉长到跨月甚至更长的时间跨度时，长程记忆便成为制约可扩展性的主要瓶颈。

8.2 脱离 Session 之后如何与 Agent 协作

前文通过双层架构将底层的多会话管理对用户隐藏，使 Session 不再是产品设计的首要问题；但当底层的执行会话不再直接对应用户界面上的会话后，如何向用户呈现任务并提供自然的交互与项目管理形式，便成为了新的问题。我们认为，以聊天窗口为中心的交互在一定程度上是路径依赖的结果。交互可能可以更多地发生在领域特化的交互界面中，例如像本文的实例那样，让 Super-agent 直接在 GitHub 的 Issue 和 Pull Request 中工作；Session 也可以出于用户的使用习惯而在界面上保留，但用户看到的一个会话在底层未必只对应一个 Session。

同时，前文主要讨论了人类在任务发起与结果验收阶段的交互；但在模型能力快速演进的过渡期内，许多任务仍需要人类在中途参与，如何在产品设计中兼顾用户对关键细节的逐步干

预与后台自主推进的效率，例如让用户在后台执行过程中随时介入、接管或与 Agent 实时协同工作，同样值得探索。

总的来看，用户所需的项目管理与交互形式与具体的工作领域和个人风格密切相关，最终形态很可能是高度定制化的；而 DeepSeek Harness 由于采用开放的架构设计，也更容易支持这种定制化。

8.3 如何划分上下文边界并进行 A2A 协作

前文讨论的始终是单个 Super-agent 实例内部的设计，默认其服务范围内的信息都汇总在同一个全局上下文中。但当使用范围从个人或小团队扩展到更大的组织时，组织在决定如何配备 Super-agent 时实际上是在划分上下文共享与隔离的边界。需要共享上下文的范围共用同一个实例，需要隔离的范围则各自独立配备。

进一步看，一旦组织内或组织间存在多个彼此隔离的实例，例如每个成员各自的个人助手与团队共用的 Super-agent 之间，此时跨边界的协同就是 Agent 间通信（A2A，不特指某个协议）。在互不共享底层上下文、各自代表不同权限边界的前提下，如何实现安全、可靠的 A2A 通信与协作，仍需进一步探索。

8.4 如何保障安全与隐私

第 3 节的事件驱动设计让 Agent 能够被更广泛的环境驱动，但也增加了接触不可信输入的风险。如果没有办法很好的解决安全与隐私问题，那么 Super-agent 只有在一个很小且可信的范围才能工作。

例如，第 5 节将 Agent Loop 部署在云端、并在各类执行设备上安装轻量 daemon，这一设计默认建立在云端与端侧环境彼此受信任的前提之上。但当云端的 Agent Loop 持续接入外部环境事件、而端侧设备上又存有私有代码、敏感数据与本地凭据时，这种跨设备连接便引入了显著的安全与隐私隐患。一旦云端的 Agent Loop 遭遇服务端入侵或提示词注入而被攻破，其连接的全部端侧设备都面临被控制的潜在风险。

此外，即使云端未被攻破，只要端侧环境中的凭据能够进入模型的上下文，正常执行过程中同样存在凭据泄露的风险；因此还需要设计相应的机制，让 Agent 能够在完全不接触凭据本身的情况下使用凭据。如果这一架构要走向更广泛的应用，就需要在能够从多样化的环境中召回上下文的同时，解决上述安全与隐私问题。

9. 结语

回顾整套设计，我们并没有提出全新的理论，而是将计算机系统中经典的工程思想应用到了 Agent 系统的构建中。具体的实现方式会随技术发展而演进，但我们认为背后的系统设计思路有更长期的参考价值。目前，基于这套思路与 DeepSeek Harness 构建的内部实例已经在持续运行，日常参与问题复现与修复、代码审查响应与工具自身的迭代。

正如开篇所述，随着大模型执行单个任务越来越少需要人工干预，限制生产力进一步提升的瓶颈正在转向人的注意力。通过双层架构、事件驱动、异步多信道交互、存算分离与外部 Watchdog，系统在底层接管了繁琐的会话管理与上下文同步，使人类得以从具体的执行细节进一步上移到更高层的决策与验收。

在这种形态下，Agent 不再只是一个被动等待指令的对话窗口，而是演进为数字环境中持续在线的自主协作者：面向个人，它可以是跨设备协助处理日常事务的个人助手；面向团队，它融入研发工作流，承接并推进具体任务。我们相信，随着模型能力与底层系统架构的共同演进，这种更自主的工作方式有望成为人机协作的常态。

回到本文的两个补丁：MSP 把“接入”从工程适配变成协议约定，使同一套架构可以不加修改地容纳新的能力与新的设备；经验层把过去的判断整理成可注入的倾向，使同一类情境不必被反复推理。前者已经在运行，后者仍在被观察。我们更愿意把后者当作一个待验证的假设来对待——正如经验本身在系统里的地位：它可以被相信一点，但必须留着被推翻的入口。

最后，回到我们为这种形态总结的那个词：**ReActive**。

把词拆开，就是三个词根：*Re*asoning（推理）、*Act*（行动）、*ively*（主动地）。合起来是一句话——**先想清楚，再动手，并且主动地去做**：不是等人问，而是被世界触发；不是排好程再执行，而是在事件到达的那一刻开始想、开始做。

ReAct 给的是前两半：推理与行动已经在一个循环里（第 3 节）。本文要补的是第三半 *ively*：如果“行动”只能由指令发起，那么再强的推理也只是被动的；只有当触发来自环境，推理与行动才真正连成一个*主动*的循环。两个补丁正落在这里——MSP 让“被触发”在协议层成立，经验让“被触发之后”不必从零想（3.5、2.3 节）。

取名当然是修辞。但一个词若能同时压住“想、做、主动”三件事，恰恰说明个人助理缺的从来不是更多推理，而是更完整的反应。这也是我们把这句话说成“ReActive is All PA Need”的原因——*双关是刻意的，判断是认真的。*

本文是在一篇公开发表的文章《Super-agent 设计探索：迈向更自主的 Agent 系统》基础上的改写：第 1 至 8 节的架构叙述、实验数据与原有插图沿用该文；两个补丁（MSP、Mem +

Exp + SKILL）、实现状态一节、新增插图与全文改写由本文作者完成。本文**与 DeepSeek 及其团队没有合作、从属或其他任何关系**，写作过程中也未与之发生交流。

Author & Contributions

本文由 Xuanhao Hu 独立完成：提出并实现了 MSP 与经验层（Mem + Exp + SKILL），并完成本文的改写与写作。文中错误由作者自负。

声明。本文为作者独立撰写的研究性文章，**与 DeepSeek 及其产品没有直接关系**，不代表 DeepSeek 的立场，也不构成对任何产品的介绍或背书。文中的基线架构叙述取自一篇公开发表的《Super-agent 设计探索》`https://mp.weixin.qq.com/s/zoS7GlReWPhfCjENGxXCdg`；MSP 与 Mem/Exp/SKILL 的设计、实现与写作由作者完成。ReActive 是一个架构主张，其中的经验层仍属实验性工作，请勿当作成熟的工程承诺。